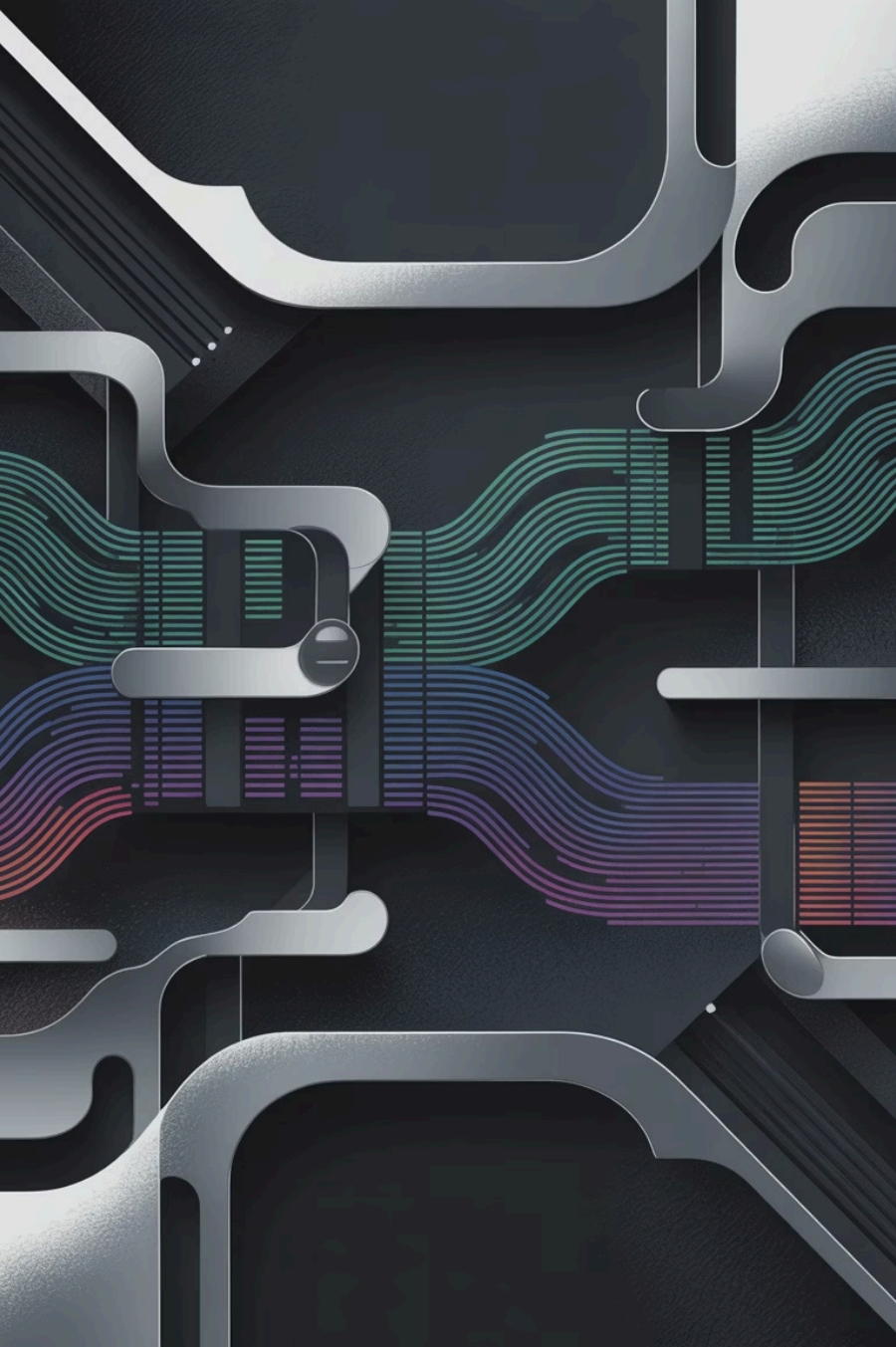




Module 12- Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Today's Learning Journey

01

JSONL Loading Fundamentals

Understanding line-delimited JSON formats and loading techniques for large-scale LLM datasets

03

Shuffling & Batching

Implementing strategies to randomize data and create efficient training batches

02

Mapping & Tokenization

Applying transformations and preparing text data for model consumption with batched operations

04

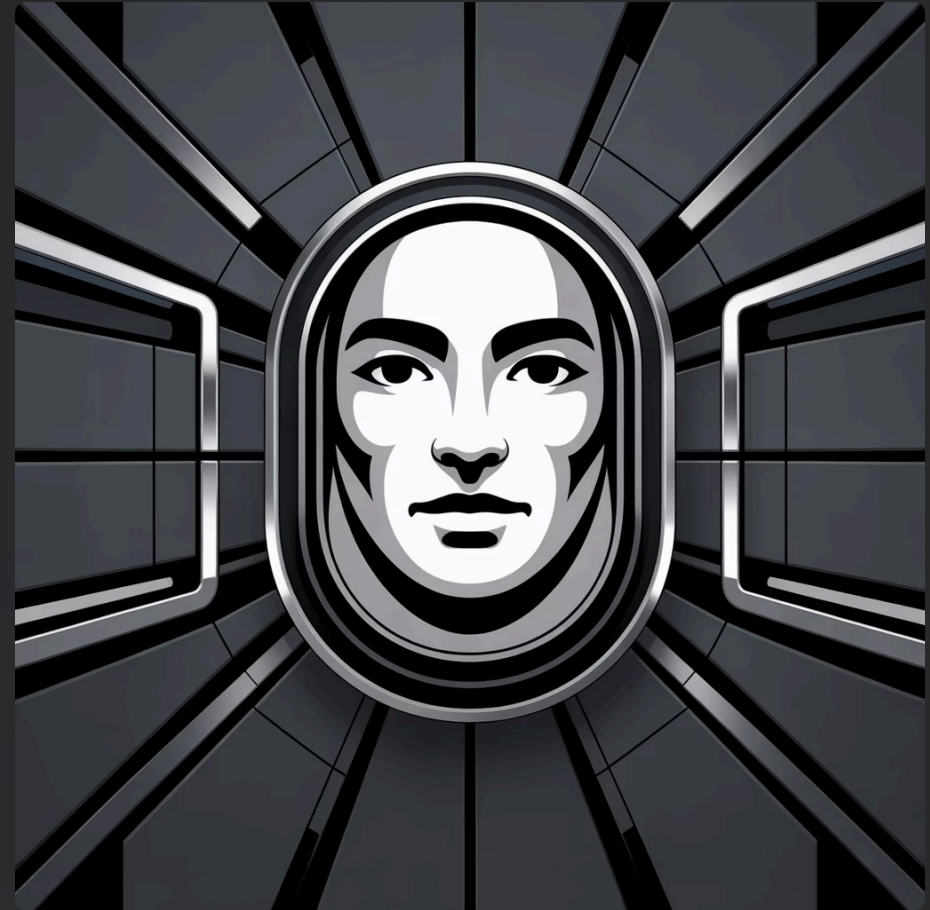
Train/Test Splitting

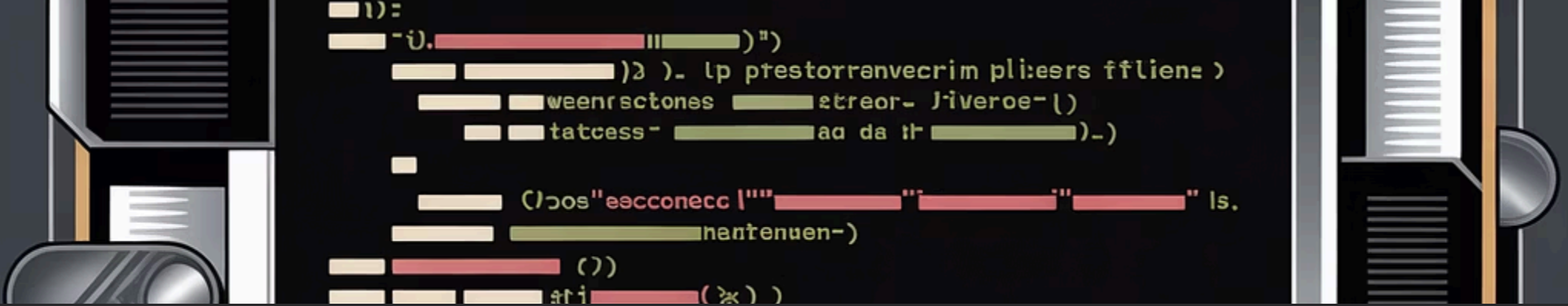
Creating robust data partitions for model evaluation and preventing overfitting

Why the Hugging Face Datasets Library?

The **datasets** library from Hugging Face has become the industry standard for loading and processing training data for LLMs. It provides memory-efficient operations through Apache Arrow, seamless integration with transformers, and built-in caching mechanisms that dramatically speed up iterative development.

Unlike traditional pandas DataFrames, datasets can handle multi-gigabyte files without loading everything into RAM. This makes it essential for production LLM pipelines where training sets routinely exceed available memory.





JSONL: The Standard Format for LLM Training Data

JSONL (JSON Lines) is a text format where each line is a valid JSON object. This format is particularly well-suited for large-scale machine learning because you can stream and process it line-by-line without loading the entire file into memory.

Each line represents a single training example, making it trivial to split, shuffle, and distribute across multiple workers. For LLM fine-tuning, JSONL files typically contain fields like "prompt", "completion", "instruction", or "text" depending on your task.

```
{"prompt": "Translate to French:", "completion": "Bonjour"}  
{"prompt": "Summarize this article:", "completion": "The key points are..."}  
{"prompt": "Answer the question:", "completion": "The answer is..."}
```

Loading JSONL with the Datasets Library

Basic Loading Pattern

The `load_dataset()` function provides a unified interface for loading data from local files, Hugging Face Hub, or custom data loaders. For JSONL files, specify the `json` data type.

The `data_files` parameter accepts file paths, glob patterns, or dictionaries mapping split names to files. This flexibility allows you to structure your training pipeline exactly as needed.

```
from datasets import load_dataset
```

```
# Load single JSONL file
```

```
dataset = load_dataset(  
    "json",  
    data_files="train.jsonl"  
)
```

```
# Load with explicit splits
```

```
dataset = load_dataset(  
    "json",  
    data_files={  
        "train": "train.jsonl",  
        "validation": "val.jsonl"  
    }  
)
```

Understanding Dataset Objects

DatasetDict

A dictionary-like container holding multiple dataset splits (train, validation, test). Each split is a separate Dataset object that can be accessed by key.

```
dataset["train"]  
dataset["validation"]
```

Dataset

The core object representing a single dataset split. Supports indexing, slicing, and iteration. Backed by Apache Arrow for memory efficiency.

```
dataset[0]  
dataset[:10]  
len(dataset)
```

Features

Schema definition describing the structure and types of data columns. Enforces type safety and enables optimized storage and processing.

```
dataset.features  
dataset.column_names
```

Exploring Your Dataset

```
from datasets import load_dataset

# Load instruction-tuning dataset
dataset = load_dataset("json", data_files="instructions.jsonl")

# Examine structure
print(dataset)
# DatasetDict({
#   train: Dataset({
#     features: ['instruction', 'input', 'output'],
#     num_rows: 52000
#   })
# })

# Inspect first example
print(dataset["train"][0])
# {'instruction': 'Classify the sentiment...',
#   'input': 'This product exceeded my expectations!',
#   'output': 'positive'}
```

Always inspect your loaded dataset to verify the schema matches expectations. Check for missing fields, unexpected data types, or malformed entries before proceeding with transformations.

The Power of .map()



Transforming Data with `.map()`

The `.map()` method is the workhorse of data preprocessing in the datasets library. It applies a transformation function to every example in your dataset, with intelligent batching and caching to maximize performance.

Key Parameters

- **function:** Transformation to apply to each example or batch
- **batched=True:** Process multiple examples at once for speed
- **batch_size:** Number of examples per batch (default 1000)
- **remove_columns:** Drop columns you no longer need
- **num_proc:** Parallel processing across CPU cores

Return Format

Your function should return a dictionary where keys are column names and values are lists (for batched) or single values (for non-batched processing).

The library handles updating the dataset schema automatically based on the returned structure.

Example: Text Preprocessing Function

```
def preprocess_text(examples):  
    """  
    Clean and normalize text data for LLM training.  
    Operates on batches for efficiency.  
    """  
  
    # Access batched inputs as lists  
    instructions = examples["instruction"]  
    inputs = examples["input"]  
    outputs = examples["output"]  
  
    # Create combined prompt-completion pairs  
    prompts = []  
    completions = []  
  
    for inst, inp, out in zip(instructions, inputs, outputs):  
        # Construct prompt with instruction and input  
        prompt = f"### Instruction:\n{inst}\n\n### Input:\n{inp}\n\n### Response:\n"  
        prompts.append(prompt)  
        completions.append(out)  
  
    # Return dictionary with new columns  
    return {  
        "prompt": prompts,  
        "completion": completions  
    }
```

Applying Transformations

Apply your preprocessing function using `.map()` with `batched=True` to process multiple examples simultaneously. This dramatically improves performance compared to row-by-row processing.

The `remove_columns` parameter cleans up your dataset by dropping original columns that are no longer needed after transformation. This reduces memory usage and simplifies downstream processing.

Results are automatically cached to disk, so re-running the same transformation is nearly instantaneous.

```
processed_dataset = dataset.map(  
    preprocess_text,  
    batched=True,  
    batch_size=1000,  
    remove_columns=[  
        "instruction",  
        "input",  
        "output"  
    ]  
)  
  
# Verify transformation  
print(processed_dataset["train"][0])  
# {  
#   'prompt': '### Instruction:\n...',  
#   'completion': 'positive'  
# }
```

Tokenization: Preparing Text for Models

Tokenization converts raw text into numerical token IDs that language models can process. Each model has its own tokenizer that splits text according to its vocabulary and special rules learned during pre-training.

The Hugging Face **transformers** library provides AutoTokenizer, which automatically loads the correct tokenizer for any model. Tokenizers handle special tokens, padding, truncation, and attention masks required by the model architecture.



Loading and Configuring a Tokenizer

```
from transformers import AutoTokenizer

# Load tokenizer for specific model
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# Configure tokenization parameters
def tokenize_function(examples):
    """
    Tokenize text with padding and truncation.
    Returns input_ids and attention_mask for model consumption.
    """
    return tokenizer(
        examples["text"],
        padding="max_length", # Pad to max_length
        truncation=True,      # Truncate sequences exceeding max_length
        max_length=512,       # Maximum sequence length
        return_tensors=None    # Return lists, not tensors (for datasets lib)
    )
```

The tokenizer returns `input_ids` (token indices), `attention_mask` (which tokens to attend to), and potentially other model-specific tensors. All of these become new columns in your dataset.

Tokenizing Your Dataset

```
from transformers import AutoTokenizer

# Initialize tokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "gpt2"
)

# Define tokenization function
def tokenize(examples):
    return tokenizer(
        examples["prompt"],
        truncation=True,
        max_length=512
    )

# Apply tokenization
tokenized = processed_dataset.map(
    tokenize,
    batched=True,
    remove_columns=["prompt"]
)
```

Batched Tokenization Benefits

Processing examples in batches allows the tokenizer to leverage optimized Rust implementations and parallel processing. For large datasets, this can reduce tokenization time from hours to minutes.

The `batched=True` parameter is crucial—it passes lists of strings to the tokenizer instead of individual strings, enabling vectorized operations.

Understanding Tokenizer Output

input_ids

List of integers representing token indices in the model's vocabulary. This is the primary input to the model.

```
[101, 2023, 2003, 1037, ...]
```

attention_mask

Binary mask indicating which tokens should be attended to (1) and which are padding (0). Essential for batched processing.

```
[1, 1, 1, 1, 0, 0, ...]
```

token_type_ids

Segment IDs for models like BERT that process multiple sequences. Distinguishes between sentence A and sentence B.

```
[0, 0, 0, 1, 1, 1, ...]
```

After tokenization, your dataset columns include these tensor fields alongside any remaining text columns. You can remove text columns to save memory before training.



CLS
SEP
PAD

Handling Special Tokens and Padding

Special Tokens

Models use special tokens to mark sequence boundaries and separate segments. Common tokens include:

- [CLS] or `<|endoftext|>` - Start of sequence
- [SEP] - Separator between segments
- [PAD] - Padding to reach uniform length
- [UNK] - Unknown/out-of-vocabulary tokens

Padding Strategies

Different padding approaches for different scenarios:

- **max_length**: Pad all sequences to specified length
- **longest**: Pad to longest sequence in batch
- **do_not_pad**: Leave sequences as-is for dynamic batching

Choose based on your training framework and memory constraints.



Shuffling & Batching Strategies

Why Shuffling Matters for Training

Shuffling randomizes the order of training examples, which is critical for stochastic gradient descent and preventing the model from learning spurious patterns based on data order. Without shuffling, models can overfit to the sequence of examples and fail to generalize.



Sequential Data

Examples ordered by topic, length, or source create biased batches



Shuffle Operation

Randomize order across the entire dataset using a seed for reproducibility



Mixed Batches

Each batch contains diverse examples, improving gradient estimates

Implementing Dataset Shuffling

```
from datasets import load_dataset

# In-memory shuffle: permutes an index over the Arrow table.
# No buffer needed — the dataset is memory-mapped from disk.
dataset = load_dataset("json", data_files="train.jsonl")
shuffled = dataset["train"].shuffle(seed=42) # reproducible

# For datasets too large to materialize, STREAM and shuffle
# with a buffer. buffer_size belongs to IterableDataset only.
streamed = load_dataset(
    "json", data_files="train.jsonl", streaming=True
)["train"]
shuffled_stream = streamed.shuffle(buffer_size=10000, seed=42)

# Verify order changed
print(dataset["train"][0])
print(shuffled[0]) # different example
```

Buffer-Based Shuffling (streaming only)

For massive datasets that don't fit in memory, use `buffer_size` to shuffle within a sliding window. This provides "good enough" randomization while streaming data from disk.

A `buffer_size` of 10,000 means 10,000 examples are loaded into memory, shuffled, then examples are drawn from the buffer while new ones are loaded.

`buffer_size` *applies to* `IterableDataset` *from* `load_dataset(..., streaming=True)`, *not to an in-memory* `Dataset`.

Creating Training Batches

Batching groups multiple examples together for parallel processing on GPUs. The datasets library integrates with PyTorch and TensorFlow to create optimized data loaders that handle batching, padding, and tensor conversion automatically.

```
from torch.utils.data import DataLoader
from transformers import DataCollatorWithPadding

# GPT-2 ships without a pad token — give it one before padding.
tokenizer.pad_token = tokenizer.eos_token

# The collator needs TOKENIZED input (input_ids/attention_mask),
# so batch the *tokenized* dataset, not the raw one.
tokenized = shuffled_dataset.map(
    tokenize, batched=True, remove_columns=["prompt"]
)

# Dynamic padding: each batch is padded to its own longest row.
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

train_dataloader = DataLoader(
    tokenized,
    batch_size=16,
    collate_fn=data_collator, # pads + converts to tensors
    shuffle=True,
)

for batch in train_dataloader:
    # dynamic padding -> N varies per batch, not a fixed 512
    print(batch["input_ids"].shape) # e.g. torch.Size([16, 87])
    break
```

We'll see more in later weeks...

Data Collators: Smart Batching



DataCollatorWithPadding

Dynamically pads sequences to the longest in each batch, minimizing wasted computation on padding tokens. Ideal for models that support variable lengths.



DataCollatorForLanguageModeling

Prepares batches for causal language modeling by creating input/label pairs. Handles masking for MLM or shifting for next-token prediction.

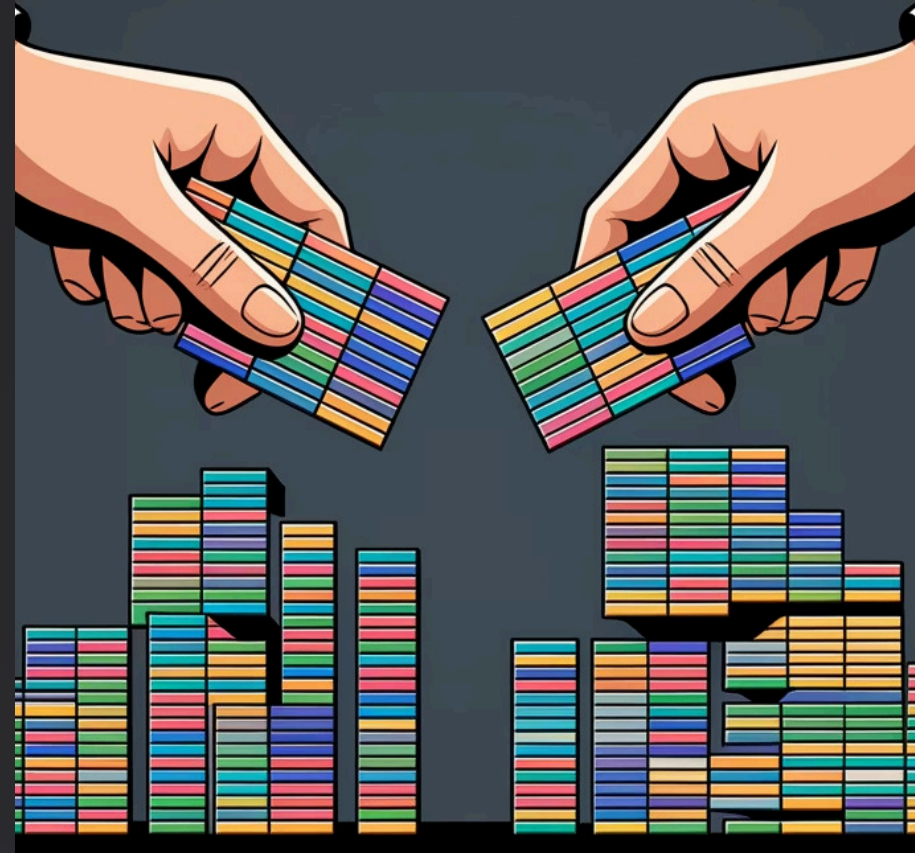


DataCollatorForSeq2Seq

Specialized collator for sequence-to-sequence tasks like translation or summarization. Manages encoder and decoder inputs separately.

Choose the appropriate collator based on your task and model architecture. Custom collators can be created by subclassing the base DataCollator class.

Train/Test Splits



The Importance of Proper Data Splits



Splitting your data into train, validation, and test sets is fundamental to building reliable machine learning systems. Each split serves a distinct purpose in the development lifecycle.

Training set: Used to update model parameters through backpropagation. Typically 70-80% of data.

Validation set: Used for hyperparameter tuning and model selection. Guides decisions without contaminating the test set. Usually 10-15% of data.

Test set: Final evaluation of model performance on unseen data. Must remain untouched until final assessment. Typically 10-15% of data.

Creating Splits with train_test_split()

```
from datasets import load_dataset

# Load full dataset
dataset = load_dataset("json", data_files="data.jsonl")

# Create train/test split (80/20)
train_test = dataset["train"].train_test_split(
    test_size=0.2,
    seed=42 # Reproducible splits
)

# Access splits
train_dataset = train_test["train"]
test_dataset = train_test["test"]

print(f"Training examples: {len(train_dataset)}")
print(f"Test examples: {len(test_dataset)}")

# Create validation split from training data
train_val = train_test["train"].train_test_split(
    test_size=0.125, # 10% of original data (0.8 * 0.125 = 0.1)
    seed=42
)

final_splits = {
    "train": train_val["train"], # 70%
    "validation": train_val["test"], # 10%
    "test": train_test["test"] # 20%
}
```

Stratified Splitting for Imbalanced Data

When your dataset has imbalanced classes or categories, stratified splitting ensures each split maintains the same class distribution as the original dataset. This prevents train/test mismatch and provides more reliable evaluation metrics.

Use the `stratify_by_column` parameter to specify which column to stratify on. This is particularly important for classification tasks with rare classes.

```
from datasets import load_dataset

dataset = load_dataset("json", data_files="sentiments.jsonl")

# JSON loads `label` as a plain int64. Stratification needs the
# finite class set, so promote it to a ClassLabel feature first.
ds = dataset["train"].class_encode_column("label")
print(ds.features["label"])
# ClassLabel(names=['0', '1', '2'])

# Now stratify: both splits keep the original class ratios.
stratified_split = ds.train_test_split(
    test_size=0.2,
    stratify_by_column="label",
    seed=42,
)

print("Train labels:", stratified_split["train"]["label"][:100])
print("Test labels:", stratified_split["test"]["label"][:100])
```

`stratify_by_column` requires a `ClassLabel` column; cast plain int/string labels with `class_encode_column(...)` first.

Best Practices for Data Splitting

Always Set a Random Seed

Use a fixed seed value to ensure reproducible splits across experiments. This allows you to compare model iterations fairly and debug issues consistently.

Split Before Preprocessing

Create train/test splits on raw data before tokenization or feature engineering to prevent information leakage from preprocessing statistics calculated on the full dataset.

Preserve Temporal Order for Time-Series

For data with temporal dependencies, don't use random splits. Instead, split chronologically so training data precedes validation and test data, mimicking real-world deployment.

Avoid Duplicate Examples Across Splits

Check for near-duplicate examples that might appear in both train and test sets. This is common in web-scraped data and can inflate performance metrics artificially.

Complete Pipeline: Putting It All Together

```
from datasets import load_dataset
from transformers import AutoTokenizer

# 1. Load JSONL data
dataset = load_dataset("json", data_files="qa_pairs.jsonl")

# 2. Create train/validation/test splits
train_test = dataset["train"].train_test_split(test_size=0.2, seed=42)
train_val = train_test["train"].train_test_split(test_size=0.125, seed=42)

splits = {
    "train": train_val["train"],
    "validation": train_val["test"],
    "test": train_test["test"]
}

# 3. Preprocess and tokenize
tokenizer = AutoTokenizer.from_pretrained("t5-small")

def preprocess_and_tokenize(examples):
    # Combine question and context
    inputs = [f"question: {q} context: {c}" for q, c in
               zip(examples["question"], examples["context"])]
    model_inputs = tokenizer(inputs, max_length=512, truncation=True)

    # Tokenize answers as labels
    labels = tokenizer(examples["answer"], max_length=128, truncation=True)
    model_inputs["labels"] = labels["input_ids"]

    return model_inputs

tokenized_splits = {
    split: splits[split].map(
        preprocess_and_tokenize,
        batched=True,
        remove_columns=splits[split].column_names
    )
    for split in splits
}

# 4. Shuffle training data
tokenized_splits["train"] = tokenized_splits["train"].shuffle(seed=42)

# 5. Create data loaders
from torch.utils.data import DataLoader
from transformers import DataCollatorForSeq2Seq

data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer)

train_dataloader = DataLoader(
    tokenized_splits["train"],
    batch_size=16,
    collate_fn=data_collator,
    shuffle=True
)

val_dataloader = DataLoader(
    tokenized_splits["validation"],
    batch_size=16,
    collate_fn=data_collator
)

# Ready for training!
print(f"Training batches: {len(train_dataloader)}")
print(f"Validation batches: {len(val_dataloader)}")
```

Common Pitfalls and Solutions

Memory Issues with Large Datasets

Problem: Dataset exceeds available RAM during loading or processing.

Solution: Use streaming mode with `load_dataset(..., streaming=True)` or enable disk caching. Process data in chunks and leverage buffer-based shuffling.

Slow Tokenization

Problem: Tokenization takes hours for large datasets.

Solution: Always use `batched=True` with `.map()`. Increase `num_proc` to use multiple CPU cores. Consider using fast tokenizers (Rust-based) instead of Python tokenizers.

Inconsistent Split Sizes

Problem: Train/test splits vary between runs, making results non-reproducible.

Solution: Always specify a fixed seed parameter in `train_test_split()`. Document the seed value in your code and experiment logs.

Data Leakage Through Preprocessing

Problem: Normalization statistics or vocabulary built on full dataset leak test set information into training.

Solution: Split data first, then apply preprocessing separately to each split. Calculate statistics only on training data and apply them to validation/test sets.

Performance Optimization Tips

Caching

The datasets library automatically caches processed data to `~/.cache/huggingface/datasets`. Subsequent runs skip recomputation for identical operations.

Use `load_from_cache_file=False` to force reprocessing when needed.

Multiprocessing

Add `num_proc=4` (or more) to `.map()` calls to parallelize processing across CPU cores. Particularly effective for tokenization and feature extraction.

Be cautious with very small datasets—overhead can exceed benefits.

Batched Operations

Always use `batched=True` with appropriate `batch_size` (typically 1000-10000). This enables vectorized operations and dramatically reduces Python interpreter overhead.

Profile your pipeline to find optimal batch size for your hardware.

These optimizations can reduce pipeline execution time from hours to minutes for typical LLM fine-tuning datasets. Invest time in profiling and optimization—it pays dividends throughout iterative development.



Thanks